

First Fit:

Pseudo Code:

```
MAX = 10

blockSize = [0] * MAX
processSize = [0] * MAX
blockAllocated = [0] * MAX
allocation = [-1] * MAX

totalInternalFrag = 0
totalExternalFrag = 0

nBlocks = int(input("Enter number of memory blocks: "))

print("Enter sizes of memory blocks:")
for i in range(nBlocks):
    blockSize[i] = int(input(f"Block {i + 1}: "))

nProcesses = int(input("\nEnter number of processes: "))

print("Enter sizes of processes:")
for i in range(nProcesses):
    processSize[i] = int(input(f"Process {i + 1}: "))
    allocation[i] = -1

for i in range(nProcesses):
    for j in range(nBlocks):
        if not blockAllocated[j] and blockSize[j] >= processSize[i]:
            allocation[i] = j
            blockAllocated[j] = 1
            totalInternalFrag += blockSize[j] - processSize[i]
            break
```

```
# Output
print("\nProcess No.\tProcess Size\tBlock No.")
for i in range(nProcesses):
    print(f"{i + 1}\t\t\t{processSize[i]}\t\t\t", end="")
    if allocation[i] != -1:
        print(allocation[i] + 1)
    else:
        print("Not Allocated")

print(f"\nTotal Internal Fragmentation: {totalInternalFrag}")

for i in range(nBlocks):
    if not blockAllocated[i]:
        totalExternalFrag += blockSize[i]

print(f"\nTotal External Fragmentation: {totalExternalFrag}")
```

Input:

Enter number of memory blocks: 5

Block 1: 100

Block 2: 500

Block 3: 200

Block 4: 300

Block 5: 600

Enter number of processes: 4

Process 1: 212

Process 2: 417

Process 3: 112

Process 4: 426

Output:

```
Enter sizes of memory blocks:
```

```
Enter sizes of processes:
```

Process No.	Process Size	Block No.
1	212	2
2	417	5
3	112	3
4	426	Not Allocated

```
Total Internal Fragmentation: 559
```

```
Total External Fragmentation: 400
```

Next Fit:

Pseudo Code:

```
MAX = 10

blockSize = [0] * MAX
processSize = [0] * MAX
blockAllocated = [0] * MAX
allocation = [-1] * MAX

totalInternalFrag = 0
totalExternalFrag = 0

nBlocks = int(input("Enter number of memory blocks: "))

print("Enter sizes of memory blocks:")
for i in range(nBlocks):
    blockSize[i] = int(input(f"Block {i + 1}: "))

nProcesses = int(input("\nEnter number of processes: "))

print("Enter sizes of processes:")
for i in range(nProcesses):
    processSize[i] = int(input(f"Process {i + 1}: "))
    allocation[i] = -1

lastAllocatedIndex = 0

for i in range(nProcesses):
    count = 0
    j = lastAllocatedIndex

    while count < nBlocks:
        if not blockAllocated[j] and blockSize[j] >= processSize[i]:
            allocation[i] = j
            blockAllocated[j] = 1

            totalInternalFrag += blockSize[j] - processSize[i]
            lastAllocatedIndex = (j + 1) % nBlocks
            break

        j = (j + 1) % nBlocks
        count += 1

# Output
print("\nProcess No.\tProcess Size\tBlock No.")
for i in range(nProcesses):
    print(f"{i + 1}\t\t\t{processSize[i]}\t\t\t", end="")
    if allocation[i] != -1:
        print(allocation[i] + 1)
    else:
        print("Not Allocated")

print(f"\nTotal Internal Fragmentation: {totalInternalFrag}")

for i in range(nBlocks):
    if not blockAllocated[i]:
        totalExternalFrag += blockSize[i]

print(f"Total External Fragmentation: {totalExternalFrag}")
```

Input:

Enter number of memory blocks: 5

Block 1: 100

Block 2: 500

Block 3: 200

Block 4: 300

Block 5: 600

Enter number of processes: 4

Process 1: 212

Process 2: 417

Process 3: 112

Process 4: 426

Output:

```
Enter sizes of memory blocks:
```

```
Enter sizes of processes:
```

Process No.	Process Size	Block No.
1	212	2
2	417	5
3	112	3
4	426	Not Allocated

```
Total Internal Fragmentation: 559
```

```
Total External Fragmentation: 400
```